

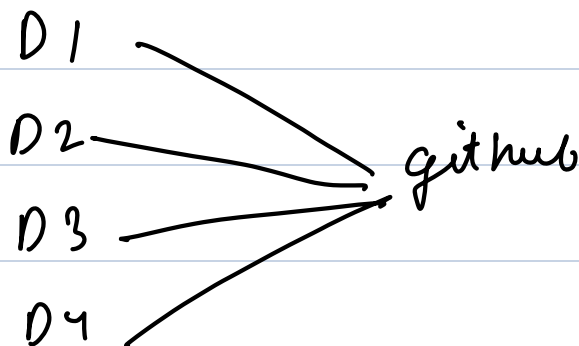
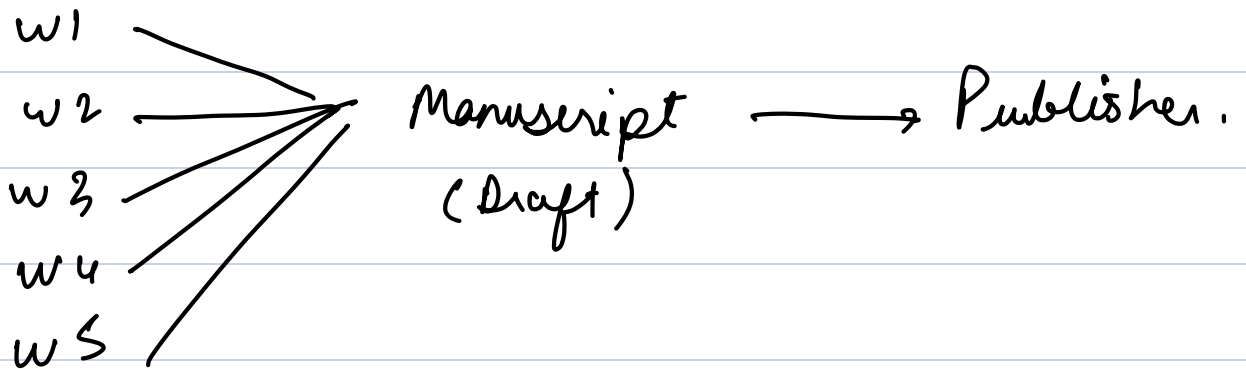
Introduction to CI/CD & Jenkins

starts at 9:05 pm

Agenda

1. CI/CD
2. Introduction to Jenkins
3. Architecture
4. Installation / GUI
5. Types of Jobs
6. Jenkins Configuration

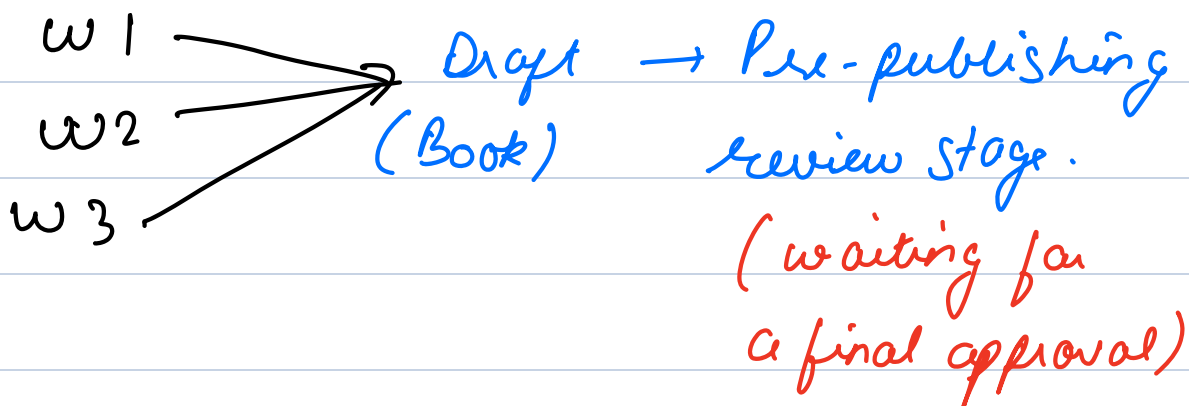
→ CI Continuous Integration.



- ① Make changes to the code
- ② Pushed to a Repository
- ③ Building. → build SC into executable
 - compiling
 - dependency mang.
 - packaging.
 - Maven or gradle
- ④ Testing.
 - unit tests
 - performance test
 - integration testing

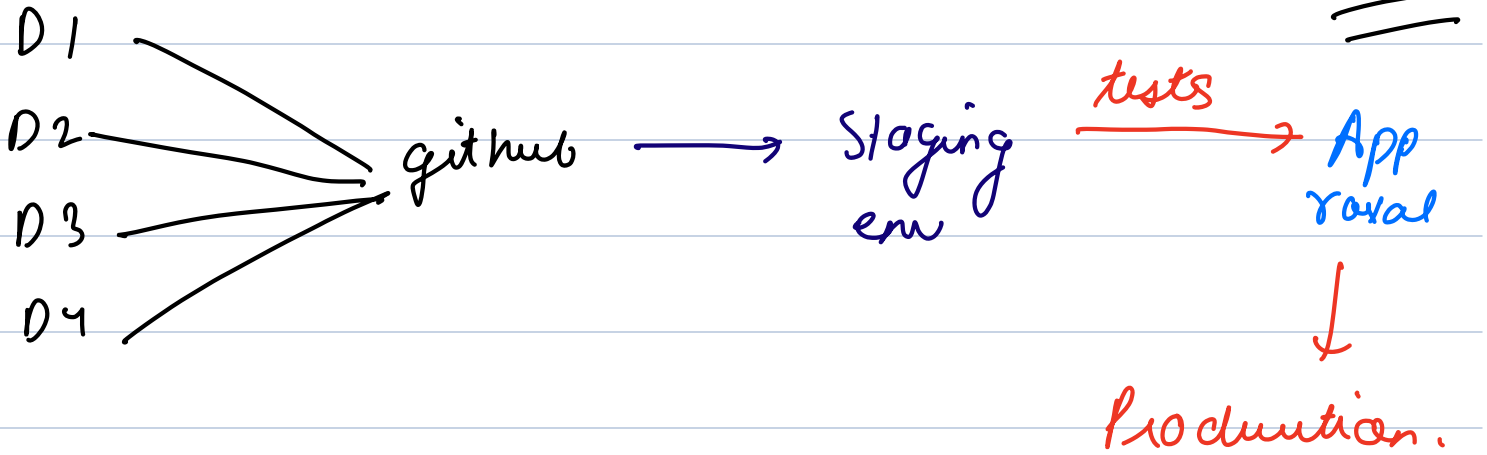
CD

Continuous Delivery.

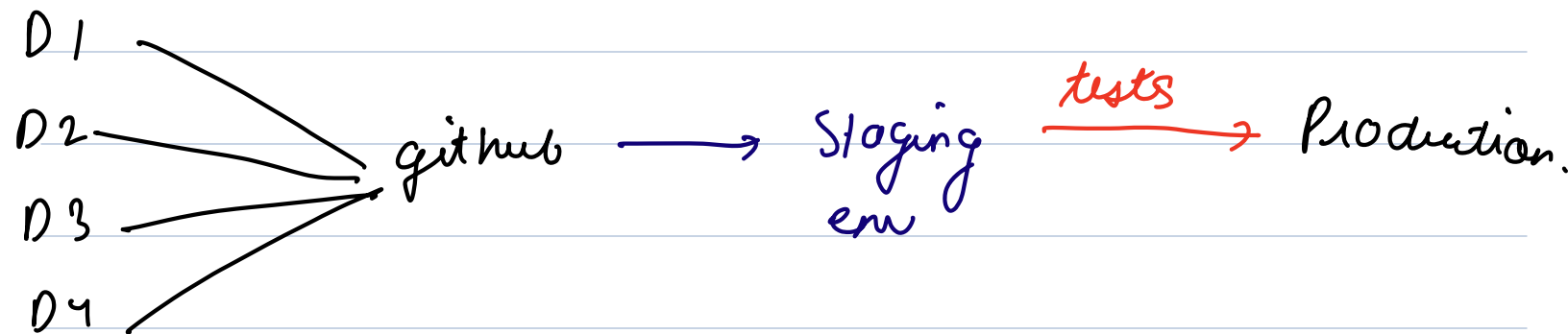
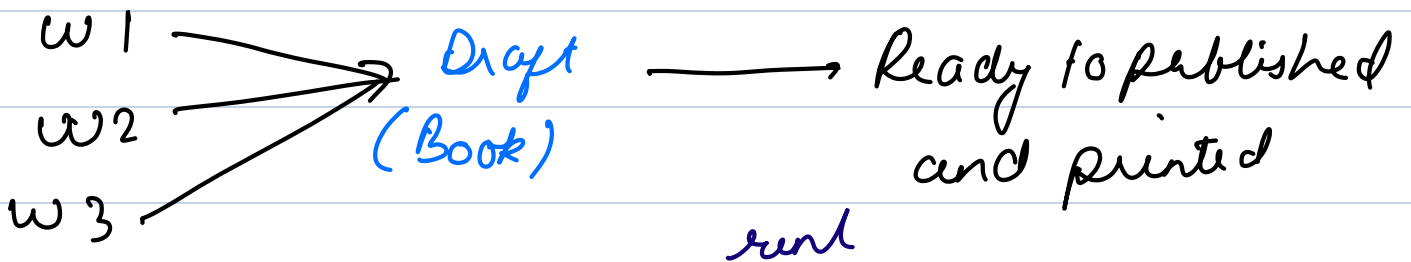


Approved

Print



Continuous Deployment



Feature	Continuous Delivery	Continuous Deployment
Definition	Ensures code is always in a deployable state and can be released anytime with a manual approval step.	Fully automated deployment; every change that passes tests is deployed to production without manual approval .
Deployment Trigger	Manual (e.g., a button click, approval step).	Automatic (no manual intervention).
Testing Required	Rigorous automated testing to ensure stability.	Even stricter automated testing, as no manual checks occur before deployment.
Risk Factor	Lower risk as human intervention allows verification before release.	Higher risk as faulty changes might reach production if tests fail to catch issues.
Use Case	Regulated industries (finance, healthcare) where approval is required.	Consumer applications (e.g., Facebook, Netflix) that require fast feature releases.

CD phases

① Release.

→ Versioning.

→ Artifacts are stored in a registry.

② Deploy.

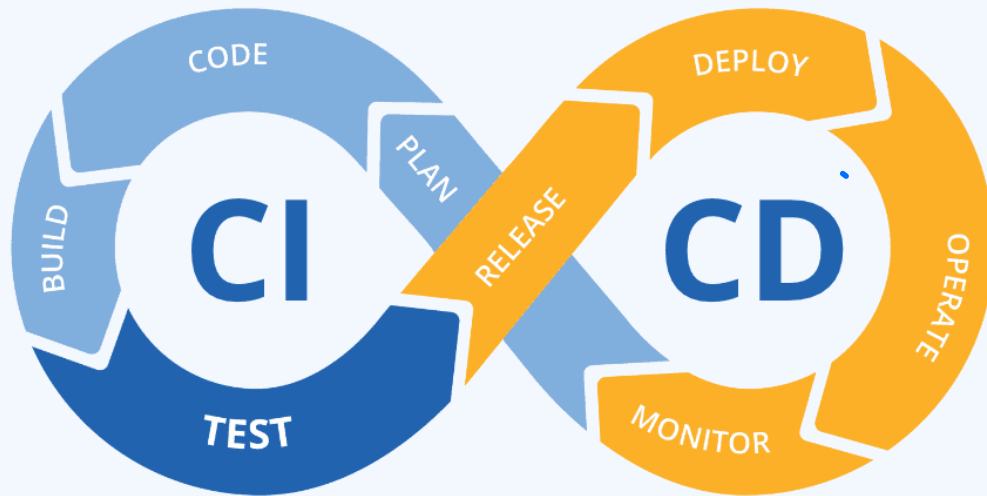
Blue, Canary, Rolling update

③ Monitor

④ Operate

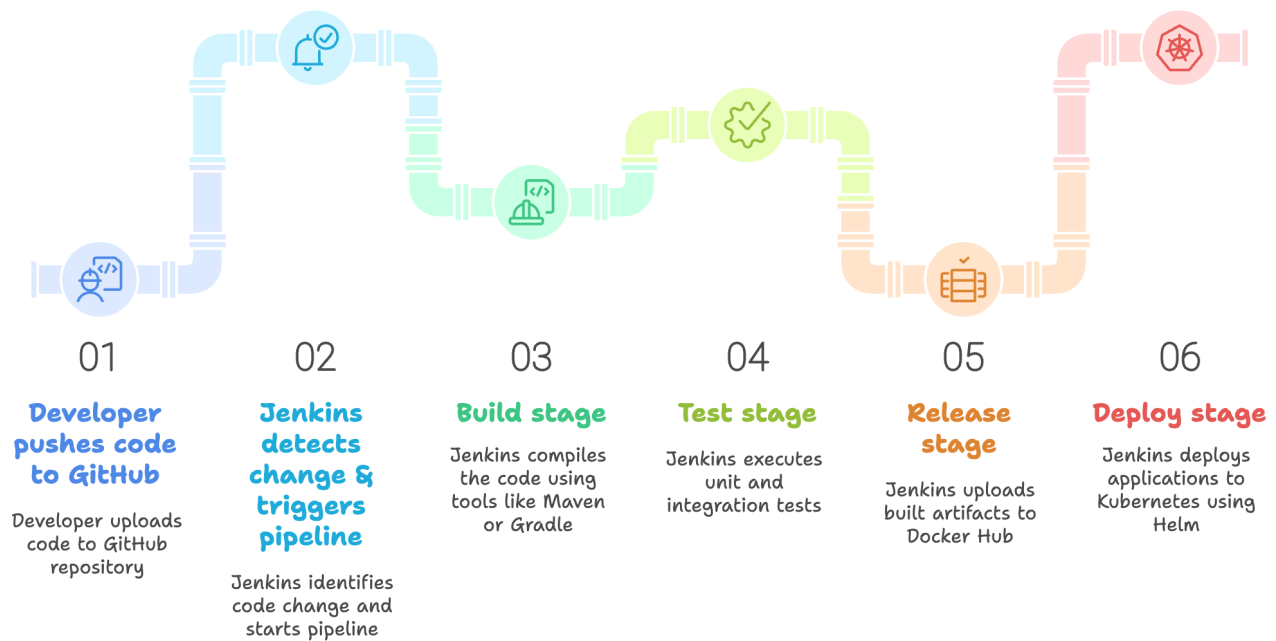
→ Autoscaling.

→ DR. Rollback Mechanisms.



Where does Jenkins come into picture?

Jenkins CI/CD Pipeline Sequence



→ What is Jenkins?

→ building
→ testing
→ deploying.

It allows developers to integrate their code frequently and ensures that software is always in a deployable state.

Why Jenkins is Still Relevant?*

1. **Massive Adoption & Ecosystem**

- Jenkins has been around for nearly **two decades**(launched in around 2004 as Hudson then renamed as Jenkins in 2011) and is deeply integrated into many organizations' CI/CD workflows.

- Over **1,800 plugins** allow it to integrate with almost any tool or cloud provider.

2. **Flexibility & Customization**

- Jenkins can support **any type of pipeline**, from legacy monolithic applications to modern microservices.

- It works with **on-premise, hybrid, and cloud** environments.

3. **Enterprise Support & Legacy Systems**

- Large enterprises still rely on Jenkins, especially in **regulated industries** where stability and security matter more than bleeding-edge technology.

4. **Open-Source & Free**

- Jenkins is **100% open-source**, meaning no licensing costs.

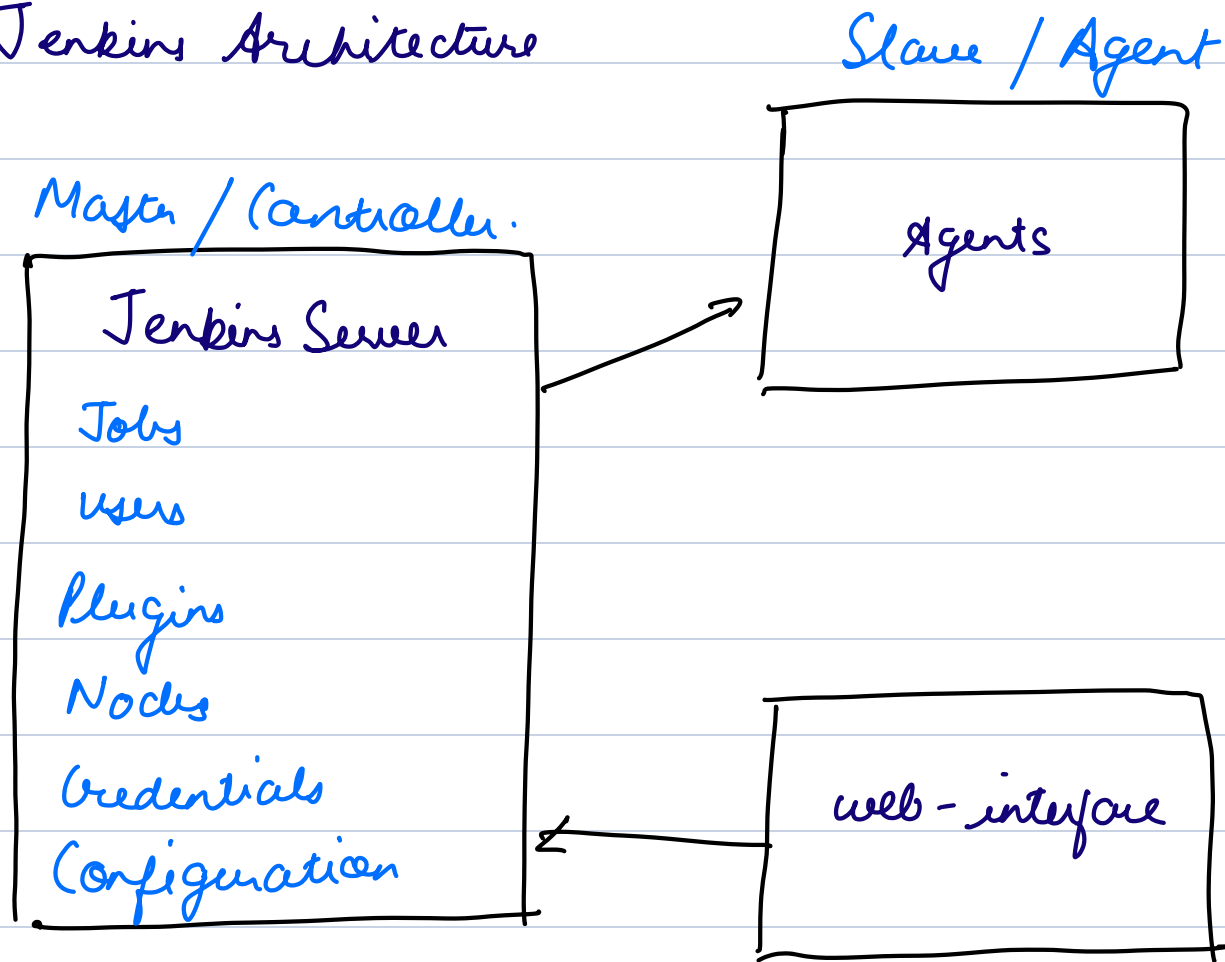
- The **large community** ensures continuous improvements, security patches, and plugin development.

5. **Easy to Set Up & Use**

- Jenkins can be installed and configured **quickly** on any machine, from a **laptop to a cloud server**.

- A simple **Java-based installation** makes it easy for teams to get started with minimal effort.

→ Jenkins Architecture



Master Responsible for.

→ Managing Jobs

→ UI

→ Scheduling Builds

→ Storing Build logs (saves history)

→ Managing Plugins.

→ slave

→ Installation.

Requirements

→ 16GB RAM

→ 4 GB

→ Java. 11 or 17. (recommended)

Install Java

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y fontconfig openjdk-17-jre
```

→ Add Jenkins key. & Repo.

```
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/
```

```
null
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian binary/" | sudo tee /etc/apt/
```

```
sources.list.d/jenkins.list > /dev/null
```

Install Jenkins

sudo apt update

sudo apt install -y jenkins

****Start and Enable Jenkins****

sudo systemctl start jenkins

sudo systemctl enable jenkins

Access GUI

http://ip:8080

****Unlock Jenkins****

Jenkins requires an ****initial admin password****.

Find it using:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Create Admin User

- Create an ****admin user**** and complete setup.

- Click ****"Start using Jenkins"**.**

Break → 10:30pm

→ Different Components of Jenkins Architecture

① Jobs

→ building the code

→ running tests

→ deploying applications or

→ even automating repetitive tasks.

Jobs define how the CI/CD processes run.

```
echo "Hello Devops Learners!"
```

```
pwd
```

```
cp $0 /var/lib/jenkins/workspace/saved-script.sh
```

} Job content

```
export NO_CLEANUP=true
```

② Users

Users

Roles → what user can do?
can't do?

1. ****Administrator**** – Full control over Jenkins (install plugins, create jobs, manage users).
2. ****Job Creator**** – Can create and modify jobs but cannot manage Jenkins settings.
3. ****Job Executor**** – Can run jobs but cannot modify them.
4. ****Read-Only User**** – Can view job statuses, logs, and configurations but ****cannot**** trigger builds or make changes.

① Create user

② Plugins. (Installed)
→ Role Based Strategy.

③ Creating the Role

④ Assigning the Role

⑤ Nodes (Agents)

⑥ Credentials.

Stores. → Place where credentials are stored.

****Types of Stores:****

- ****System****: Used for credentials that are globally available to Jenkins and its components.
- ****User**** (if enabled): Each user has a personal store for credentials.
- ****Folder-specific stores****: If you are using folders in Jenkins, each folder can have its own credential store.
- ****Job-specific stores****: Some plugins allow credentials to be attached at the job level.

→ Domains → organising, grouping.

Aspect	Domain	Store
Purpose	Categorizes and scopes credentials.	Stores and holds credentials in the Jenkins environment.
Scope	Defines which credentials are available to specific jobs or environments.	A centralized repository where all credentials are stored.
Use Case	Segregates credentials based on projects, teams, or environments.	Manages the creation, modification, and deletion of credentials.
Example	"dev-domain", "prod-domain" (for different environments).	Global store or external credential stores.

```
withCredentials([usernamePassword(credentialsId: 'vedant', usernameVariable: 'USERNAME', passwordVariable:
```

```
'PASSWORD'))] {
```

```
    echo "Username: ${USERNAME}"
```

```
    echo "Password: ${PASSWORD}"
```

```
}
```



vedant

vedant

→ Project

- ① Dev push code
- ② webhook triggers Jenkins pipeline
- ③ Jenkins:
 - Clone repo
 - Build the JAR file
 - Create & push Docker image
 - Deploy the K8s.
- ④ K8s rolling update.